

A.Tejaswini-192471043

Q101. Coin Change vs Unbounded Knapsack (DP Formulation Comparison) Given denominations and capacity, compare Coin Change (minimum coins) with Unbounded Knapsack (maximum value) using DP. Analyze differences in objective functions and state transitions. Evaluate recurrence relations and table initialization. Discuss time and space complexity. Examine scalability with larger inputs. Generate insights on modeling reuse of items in DP.

Ans:

#Q101. Coin Change vs Unbounded Knapsack

Dynamic Programming Comparison

```
def coin_change_min_coins(coins, amount):
```

```
    INF = 999999
```

```
    dp = [INF] * (amount + 1)
```

```
    dp[0] = 0
```

```
    for current_amount in range(1, amount + 1):
```

```
        for coin in coins:
```

```
            if coin <= current_amount:
```

```
                dp[current_amount] = min(
```

```
                    dp[current_amount],
```

```
                    dp[current_amount - coin] + 1
```

```
                )
```

```
            if dp[amount] == INF:
```

```
                return -1
```

```
            return dp[amount]
```

```
def unbounded_knapsack(weights, values, capacity):
```

```
    dp = [0] * (capacity + 1)
```

```
    for current_capacity in range(1, capacity + 1):
```

```
        for i in range(len(weights)):
```

```
            if weights[i] <= current_capacity:
```

```
                dp[current_capacity] = max(
```

```
                    dp[current_capacity],
```

```
                    dp[current_capacity - weights[i]] + values[i]
```

```

)
return dp[capacity]
# Coin Change Example
coins = [1, 2, 5]
amount = 11
min_coins = coin_change_min_coins(coins, amount)
print("Coin Change Problem")
print("Coins:", coins)
print("Amount:", amount)
print("Minimum coins required:", min_coins)
# Unbounded Knapsack Example
weights = [2, 3, 4]
values = [40, 50, 60]
capacity = 8
max_value = unbounded_knapsack(weights, values, capacity)
print("\nUnbounded Knapsack Problem")
print("Weights:", weights)
print("Values:", values)
print("Capacity:", capacity)
print("Maximum value:", max_value)

```

Q102. Dice Throw vs Binomial Coefficient (Recurrence Relation Analysis) Compare Dice Throw counting problem with Binomial Coefficient computation. Analyze similarities in recursive definitions and DP tabulation. Identify differences in constraints and boundary conditions. Evaluate computational complexity. Discuss memory optimization. Generate insights on combinatorial DP techniques.

Ans:

Q102. Dice Throw vs Binomial Coefficient

Dynamic Programming Recurrence Comparison

```

def dice_throw(num_sides, num_dice, target):
    dp = [[0 for _ in range(target + 1)] for _ in range(num_dice + 1)]

    dp[0][0] = 1

    for dice in range(1, num_dice + 1):
        for current_sum in range(1, target + 1):
            for face in range(1, num_sides + 1):
                if current_sum - face >= 0:
                    dp[dice][current_sum] += dp[dice - 1][current_sum - face]

    return dp[num_dice][target]

```

```

def binomial_coefficient(n, r):
    dp = [[0 for _ in range(r + 1)] for _ in range(n + 1)]

    for i in range(n + 1):
        for j in range(min(i, r) + 1):
            if j == 0 or j == i:
                dp[i][j] = 1
            else:
                dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j]

    return dp[n][r]

```

```
# Dice Throw Example
```

```
num_sides = 6
```

```
num_dice = 2
```

```
target = 7
```

```
ways = dice_throw(num_sides, num_dice, target)
```

```
print("Dice Throw Problem")
```

```
print("Number of sides:", num_sides)
```

```
print("Number of dice:", num_dice)
```

```
print("Target sum:", target)
```

```
print("Number of ways:", ways)
```

```
# Binomial Coefficient Example
```

```
n = 5
```

```
r = 2
```

```
result = binomial_coefficient(n, r)
```

```
print("\nBinomial Coefficient Problem")
```

```
print("n:", n)
```

```
print("r:", r)
```

```
print("nCr:", result)
```